## Documento técnico del sistema Don Pan — GA8-220501096-AA1-EV02

Módulos integrados. Aprendiz: Nikolás MV. Ficha: 3336171. Programa: Análisis y Desarrollo de Software.

Descripción de seis módulos PHP, su integración y los resultados de pruebas disponibles.

Los resultados de las pruebas corresponden al laboratorio de EV01. Para esta evidencia no volví a ejecutarlas.

## 1. Introducción

En este documento explico cómo se relacionan los módulos PHP de Don Pan, qué datos reciben y qué resultados producen. Para la entrega organicé una copia del código y sus documentos, sin cambiar el proyecto original ni el laboratorio de EV01.

La evidencia reúne la descripción del sistema, las instrucciones técnicas y los resultados de 37 pruebas unitarias y 10 casos de integración realizados en EV01. Las matrices y los anexos permiten consultar esos registros.

## 2. Objetivos

Objetivo general: explicar y entregar los módulos integrados de Don Pan, su código, los requisitos de configuración y los resultados de las pruebas disponibles.

Objetivos específicos: describir las entradas y salidas de seis módulos, explicar sus relaciones y la base de datos, documentar el ambiente usado en EV01 y presentar los resultados junto con un acta para revisión y aceptación.

## 3. Descripción general del sistema

Don Pan es una aplicación académica para gestionar operaciones de una panadería: catálogo, entradas de inventario, ventas, anulación e indicadores. La sesión identifica al empleado y su rol, dueño o empleada.

La venta relaciona productos, turno y empleado; el cliente es opcional. El stock se actualiza junto con detalles y movimientos dentro de una transacción. La anulación conserva la venta y registra responsable y fecha funcional.

El código también incluye clientes, compras, insumos y recetas. Sus funciones no quedaron cubiertas por los casos seleccionados. React no forma parte de esta entrega.

## 4. Alcance y requerimientos

Utilizo los identificadores N1–N7 para relacionar las necesidades del sistema con sus módulos y pruebas. Se basan en el código y en la documentación de EV01; no sustituyen un acta de aprobación de requerimientos.

| ID | Necesidad | Módulo y evidencia |
|---|---|---|
| N1 | Acceso de empleados y autorización | Autenticación; AutorizacionTest; I7 e I8 |
| N2 | Catálogo y estado activo | Productos; I4 e I10 |

| ID | Necesidad | Módulo y evidencia |
|---|---|---|
| N3 | Entradas trazables | Inventario; I1 e I8 |
| N4 | Venta sin stock negativo ni cambios parciales | Ventas; ReglasVentaTest; I2–I4 |
| N5 | Anulación única con responsable | Anulación; I5–I7 |
| N6 | Indicadores de ventas vigentes | Indicadores; I9 |
| N7 | Consulta API autenticada | API productos; I10 |

La aprobación de los requerimientos y la aceptación formal todavía no están documentadas.

## 5. Tecnologías utilizadas

| Componente | Versión o condición comprobada |
|---|---|
| PHP | 8.2.12 CLI en los registros; mysqli y mbstring para integración. |
| MariaDB | 11.4.13 propia del laboratorio; 14 tablas; puerto 13348. |
| Servidor web de pruebas | PHP CLI 127.0.0.1:18088; no Apache en las pruebas registradas. |
| Node.js | 24.16.0 documentado en EV01; fetch y cliente MariaDB en integración. |
| PHPUnit | 11.5.57; PHAR excluido de entrega, hash oficial conservado en manual. |
| Git | 2.56.0.windows.1 observado; rama main de EV01. |
| Visual Studio Code | Uso demostrado por las capturas; versión exacta no acreditada en ellas. |
| Producción | Sin despliegue público ni URL de módulos comprobados. |

La aplicación es PHP procedural con mysqli, HTML, CSS y JavaScript de interfaz. No se incorporan ORM ni framework MVC. El esquema SQL utiliza claves e índices relacionales.

## 6. Arquitectura del software

La separación por carpetas organiza responsabilidades, pero las vistas public/ incluyen consultas SQL; no es una arquitectura estrictamente MVC.

| Componente | Responsabilidad | Dependencias |
|---|---|---|
| public/ | Presentar formularios y consultas HTML | auth, CSRF, conexión y plantillas compartidas |

| Componente | Responsabilidad | Dependencias |
|---|---|---|
| actions/ | Procesar POST y transacciones, redirigir | reglas, conexión, sesión y helpers |
| api/ | Responder JSON y comprobar método/sesión | bootstrap, auth, CSRF y conexión |
| includes/ | Políticas comunes y presentación compartida | reglas puras; sesión en auth |
| config/ | Conectar solo al laboratorio validado | mysqli e identidad SQL |
| tests/ | Unitarias puras y ejecutor de integración | PHPUnit o infraestructura aislada histórica |

Flujo: formulario → acción → reglas compartidas → transacción mysqli → MariaDB → redirección → consulta actualizada. La API aplica controles comunes y devuelve JSON. Las unitarias cargan las mismas reglas sin abrir BD o HTTP.

## 7. Estructura del proyecto

| Ruta | Contenido entregado |
|---|---|
| actions/; api/; includes/; public/ | Código PHP, CSS, JS y logo del laboratorio |
| config/db.php | Conexión con comprobaciones conservadas sin cambios |
| sql/schema.sql | DDL de 14 tablas, sin datos ni USE del original |
| tests/; phpunit.xml | Fuentes de pruebas y configuración PHPUnit |
| scripts/ | Ejecutor de pruebas y router históricos |
| docs/resultados/ | XML, JSON y salidas auténticas de EV01 |
| docs/ | Documento técnico, acta, manual, ambiente y matrices EV02 |
| evidencias/ | Cuatro PNG reales reutilizados de EV01 |
| config/CONFIGURACION_EJEMPLO.json | Plantilla informativa sin secretos; la aplicación no la carga |

El paquete contiene el código PHP, que necesita un intérprete para ejecutarse. No requiere un archivo .exe ni una compilación de React. Las bases, sesiones, herramientas y credenciales se preparan por separado y no se incluyen.

## 8. Documentación de módulos

A continuación describo seis módulos del sistema. En cada uno indico sus archivos, funcionamiento, pruebas disponibles y aspectos que todavía no se han comprobado.

### 8.1. Autenticación y permisos

| Elemento | Descripción |
|---|---|
| Objetivo | Identificar al empleado activo y restringir operaciones por sesión y rol. |
| Funcionalidades existentes | Login PHP y JSON, consulta de sesión, cierre de sesión y comprobación del rol dueño. |
| Archivos | public/index.php; public/logout.php; actions/login_action.php; api/login.php; api/session.php; api/logout.php; includes/auth.php; includes/reglas.php; includes/csrf.php; api/bootstrap.php. |
| Datos de entrada | Formulario: usuario, password y csrf_token. API login: JSON usuario/password y cabecera CSRF de la sesión. Cookie de sesión en el navegador. |
| Validaciones | POST y CSRF; consulta de empleado activo; password_verify; regeneración del ID de sesión. API: campos obligatorios y límites de longitud. Dueño: sesión con id_empleado y rol dueño. |
| Procesamiento | Busca empleados mediante consulta preparada, verifica el hash y guarda id_empleado, nombre y rol en sesión; las comprobaciones de acceso se reutilizan antes de las operaciones protegidas. |
| Datos de salida | Formulario: redirección 302 a ventas o error. API: JSON público; 401 credenciales o sesión; 422 validación; 429 bloqueo. No devuelve password_hash. |
| Relación con otros módulos | Protege productos, inventario, ventas, anulación e indicadores. requireOwner restringe anulación y reportes. |
| Pruebas y resultados | AutorizacionTest: 6 casos. CsrfTest: 11 casos. I7 e I8 aprobados. El inicio de sesión se utiliza para preparar clientes en la integración, sin ser un caso I independiente. |
| Limitaciones | Bloqueo de 5 intentos y 60 segundos ligado a la sesión; puede evadirse con otra sesión. El límite de intentos no se conserva entre sesiones. La redirección relativa de requireOwner se conserva; el rechazo se comprobó sin seguirla. |

### 8.2. Productos

| Elemento | Descripción |
|---|---|
| Objetivo | Administrar el catálogo y consultar precio y existencias. |
| Funcionalidades existentes | Consulta y búsqueda por nombre/código, activos por defecto, opción de ver inactivos, alta, cambio de precio/stock mínimo, baja lógica y reactivación; API de lectura. |
| Archivos | public/productos.php; public/inventario.php; actions/producto_action.php; actions/producto_editar_action.php; actions/producto_estado_action.php; api/productos.php; config/db.php. |
| Datos de entrada | GET q e inactivos. Alta: nombre, precio, stock, stock_minimo, id_categoria. Edición: id_producto, precio, stock_minimo. Estado: id_producto, accion=baja/reactivar. POST con CSRF. |
| Validaciones | Sesión; POST y CSRF en acciones; alta con nombre no vacío, precio positivo y stock no negativo; edición con ID/precio positivos y mínimo no negativo; estado dentro de lista permitida. |
| Procesamiento | Consultas preparadas cuando hay parámetros. Alta transaccional y movimiento inicial cuando stock > 0; cambios de estado conservan historial. API aplica activo=1. |
| Datos de salida | HTML de catálogo y redirecciones de éxito/error; GET api/productos.php devuelve ok y productos con id_producto, nombre, precio, stock_actual, stock_minimo y codigo_barra. |
| Relación con otros módulos | Alimenta ventas e inventario y aporta las existencias a indicadores. |
| Pruebas y resultados | I10: producto activo visible, inactivo excluido y acceso sin sesión HTTP 401. I4 rechaza vender un inactivo. No hay resultados que comprueben todas las operaciones de alta, edición y baja del catálogo. |
| Limitaciones | Algunas acciones convierten los valores recibidos antes de comprobarlos. El alta convierte valores antes de validar; estas comprobaciones son diferentes de las reglas de cantidades utilizadas en las ventas. |

### 8.3. Inventario de productos

| Elemento | Descripción |
|---|---|
| Objetivo | Registrar entradas y mantener su trazabilidad. |
| Funcionalidades existentes | Consulta de existencias y registro de entrada positiva con movimiento asociado. |
| Archivos | public/inventario.php; actions/inventario_action.php; movimientos_inventario en sql/schema.sql. |
| Datos de entrada | id_producto, cantidad y csrf_token por POST; sesión autenticada. |
| Validaciones | Sesión, POST, token válido, ID y cantidad positivos tras conversión a entero. |
| Procesamiento | Transacción que incrementa productos.stock_actual e inserta un movimiento de entrada; commit o rollback. |
| Datos de salida | Redirección 302 con ok/error y stock actualizado en la base de laboratorio. |
| Relación con otros módulos | Las ventas descuentan el mismo stock y la anulación lo restituye mediante movimientos. |
| Pruebas y resultados | I1: entrada de 5, stock 10 a 15 y movimiento. I8: CSRF ausente/falso, HTTP 403 sin escritura. Ambos aprobados. |
| Limitaciones | No se ha probado la concurrencia y algunas entradas se convierten a números antes de validarse. No se ejecutaron pruebas nuevas de EV02. |

### 8.4. Ventas

| Elemento | Descripción |
|---|---|
| Objetivo | Registrar venta, detalle y salida de inventario de forma transaccional. |
| Funcionalidades existentes | Selección de cantidades, cliente opcional, método de pago y cálculo servidor del total; asignación del turno del empleado. |
| Archivos | public/ventas.php; public/js/venta-total.js; actions/venta_action.php; includes/reglas.php; includes/helpers.php; config/db.php. |
| Datos de entrada | cantidad[id_producto], id_cliente opcional, metodo_pago y csrf_token; empleado de sesión con turno. |
| Validaciones | Sesión y CSRF; cantidades enteras positivas hasta 2147483647; omisión solo de casillas vacías/cero; ID válido, producto existente y activo y stock suficiente. Pago fuera de lista se sustituye por efectivo. |
| Procesamiento | Abre transacción, crea cabecera, lee producto FOR UPDATE, valida línea, inserta detalle, descuenta stock, registra salida y actualiza total; revierte todo ante excepción. |
| Datos de salida | Venta/detalle/movimiento persistidos y redirección 302; ante fallo, rollback y redirección genérica de error. |
| Relación con otros módulos | Depende de empleados/turnos, clientes opcionales y productos; alimenta historial, anulación e indicadores. |
| Pruebas y resultados | ReglasVentaTest: 20 casos aprobados. I2: dos unidades, total 2501 y stock 15 a 13. I3: rollback tras una primera línea válida. I4: inactivo rechazado. |
| Limitaciones | El cálculo PHP usa float pese al almacenamiento DECIMAL. No se midió carga o concurrencia; FOR UPDATE es una propiedad observada del código, no un resultado de prueba concurrente. |

### 8.5. Anulación de ventas

| Elemento | Descripción |
|---|---|
| Objetivo | Anular una venta una sola vez, restituir stock y conservar auditoría. |
| Funcionalidades existentes | Consulta de historial y anulación restringida al dueño; prevención de restitución duplicada. |
| Archivos | public/historial_ventas.php; actions/anular_venta_action.php; includes/auth.php; includes/reglas.php; includes/csrf.php. |
| Datos de entrada | id_venta y csrf_token por POST; sesión del dueño. |
| Validaciones | requireLogin, método POST, csrfVerify y requireOwner antes de getConnection; ID positivo; existencia y estado no anulado. |
| Procesamiento | Transacción con bloqueo FOR UPDATE de la venta; obtiene detalles, repone cantidades, inserta movimientos y marca anulada, anulada_por y fecha_anulacion. Rollback ante error. |
| Datos de salida | Redirección 302; venta marcada y stock restituido una sola vez, sin borrado de historial. |
| Relación con otros módulos | Integra venta, productos, movimientos y empleado responsable; indicadores excluyen ventas anuladas. |
| Pruebas y resultados | I5: stock 13 a 15 y auditoría. I6: segunda anulación rechazada sin reponer. I7: empleada rechazada con datos intactos. Todos aprobados. |
| Limitaciones | No se ejecutó prueba de dos anulaciones simultáneas. La política de autorización sí tiene 6 pruebas puras compartidas con el código. |

### 8.6. Indicadores y reportes

| Elemento | Descripción |
|---|---|
| Objetivo | Consultar ventas vigentes y existencias para seguimiento del negocio. |
| Funcionalidades existentes | Dashboard diario, totales, productos/stock bajo, últimas ventas, reportes por período y cinco productos más vendidos; API resumen. |
| Archivos | public/dashboard.php; public/reportes.php; api/resumen.php; includes/auth.php; config/db.php. |
| Datos de entrada | Sesión; reportes requiere dueño y GET desde/hasta con formato YYYY-MM-DD. |
| Validaciones | Autenticación; rol dueño en reportes; fechas con patrón básico y orden corregido; API solo GET y sesión. |
| Procesamiento | Agrega ventas con anulada=0. Dashboard/API cuentan productos activos. Reportes agrega detalle por producto y período con consultas preparadas. |
| Datos de salida | HTML de indicadores/reportes; API ok, ventasHoy, totalHoy, productos y stockBajo. |
| Relación con otros módulos | Consume ventas, detalle_ventas, productos y empleados; refleja el efecto de venta/anulación. |
| Pruebas y resultados | I9: totales aumentan con la venta y vuelven al valor inicial tras anular; dashboard y reportes respondieron 200. Ese resultado no comprueba todos los filtros ni la presentación de la pantalla. |
| Limitaciones | La lista de alertas de stock de public/reportes.php no filtra activo=1; las últimas ventas del dashboard tampoco excluyen anuladas. No se equiparan estas listas a los indicadores agregados. Fechas: patrón, no validación completa del calendario. |

## 9. Integración entre componentes

El formulario de ventas envía cantidades; la acción valida sesión y CSRF y normaliza el conjunto con reglas compartidas. Cada producto se lee bajo bloqueo y se valida antes de insertar detalles, modificar stock y registrar movimientos.

| Paso registrado | Antes | Después | Evidencia |
|---|---|---|---|
| Entrada de inventario | Stock 10 | Stock 15 y movimiento +5 | I1 |
| Venta de dos unidades | Stock 15 | Stock 13; total 2501 | I2 |
| Venta con segunda línea inválida | Estado inicial de transacción | Sin cambios parciales | I3 |

| Paso registrado | Antes | Después | Evidencia |
|---|---|---|---|
| Anulación del dueño | Stock 13; venta vigente | Stock 15; venta anulada y auditada | I5 |
| Reintento de anulación | Venta anulada | Sin segunda restitución | I6 |

Indicadores: el total y conteo aumentan con la venta y regresan tras anular. El catálogo JSON conserva activos y rechaza una consulta sin sesión. No se documenta concurrencia, carga ni aceptación de usuario final.

## 10. Diseño de base de datos

El diseño se puede consultar en sql/schema.sql y contiene 14 tablas. Las pruebas de EV01 utilizaron MariaDB 11.4.13. Para preparar esta entrega no accedí a la base de datos ni ejecuté el esquema.

| Tabla | PK | Referencias |
|---|---|---|
| turnos | id_turno | Sin FK declarada |
| empleados | id_empleado | turnos.id_turno |
| categorias | id_categoria | Sin FK declarada |
| productos | id_producto | categorias.id_categoria |
| clientes | id_cliente | Sin FK declarada |
| ventas | id_venta | empleados.id_empleado; turnos.id_turno; clientes.id_cliente; empleados.id_empleado |
| detalle_ventas | id_detalle | ventas.id_venta; productos.id_producto |
| movimientos_inventario | id_movimiento | productos.id_producto |
| insumos | id_insumo | Sin FK declarada |
| recetas | id_receta | productos.id_producto; insumos.id_insumo |
| proveedores | id_proveedor | Sin FK declarada |
| compras | id_compra | proveedores.id_proveedor; empleados.id_empleado; empleados.id_empleado |
| detalle_compras | id_detalle | compras.id_compra; insumos.id_insumo |
| movimientos_insumos | id_movimiento | insumos.id_insumo; empleados.id_empleado |

Productos: precio DECIMAL(10,2), stock_actual, stock_minimo y activo. Ventas: total, metodo_pago, anulada, anulada_por y fecha_anulacion. Los detalles guardan cantidad, precio unitario y subtotal. Movimientos conservan tipo, cantidad, motivo y fecha funcional.

Índices de fecha y anulación facilitan consultas. El esquema no incluye semillas ni contraseñas. Las claves foráneas protegen relaciones; algunas tienen ON DELETE CASCADE según DDL, aunque las acciones seleccionadas conservan historial mediante baja/anulación lógica.

## 11. Seguridad y validaciones

Las acciones seleccionadas exigen sesión y CSRF antes de modificar. La anulación comprueba dueño antes de conectarse o consultar una venta. Las reglas puras de rol, token y cantidades son llamadas por producción y pruebas.

Las ventas y anulaciones utilizan transacciones y bloqueos. Las consultas con parámetros usan prepare/bind_param. Los formularios y varias salidas usan htmlspecialchars; no se afirma una auditoría exhaustiva de toda salida HTML.

config/db.php comprueba ruta real, puerto 13348, datadir, DATABASE() e identidad ga8_app@127.0.0.1. Obtiene la clave de GA8_DB_PASSWORD, no de un literal. La infraestructura histórica protege claves con DPAPI, excluidas del paquete.

Límites reales: bloqueo de login ligado a sesión; cálculo de dinero con float en PHP; validación por conversión en algunas acciones; redirecciones relativas heredadas; listas de reportes y dashboard que difieren de los filtros agregados. No se cambió funcionalidad para resolverlos.

## 12. Configuración y ejecución

Para esta evidencia organicé el código y la documentación del sistema. Los resultados de las pruebas corresponden al laboratorio utilizado en EV01. La copia de EV02 todavía no tiene una base de datos ni un servidor preparados para ejecutarse.

El manual explica qué necesita PHP para ejecutar las pruebas unitarias. El programa de integración, en cambio, solo admite la carpeta de EV01: no está listo para ejecutarse en EV02 y no debe usarse el laboratorio anterior para eludir esa restricción.

Para probar el sistema en EV02 primero se debe autorizar la adaptación del ejecutor y preparar una instancia independiente con datos ficticios. Su directorio .runtime/mariadb/data debe pertenecer a EV02. No se deben reutilizar las bases de EV01 o Don Pan; cambiar solo la contraseña no garantiza esa separación.

El comando de inicio del manual es una referencia para cuando el ambiente de EV02 esté preparado y verificado. Necesita una credencial temporal y un directorio propio de sesiones. No se ha ejecutado en esta copia ni requiere intervenir Apache o MariaDB de XAMPP.

## 13. Pruebas y resultados

Los archivos de resultados se conservaron sin cambios. PHPUnit registró 37 pruebas y 37 aserciones, sin fallos ni errores. Estas pruebas comprueban reglas en memoria y no necesitan base de datos, servidor HTTP ni sesiones reales. No se midió el porcentaje de cobertura.

| Clase | Casos | Reglas |
|---|---|---|
| AutorizacionTest | 6 | Permiso del dueño y sesiones incompletas |
| CsrfTest | 11 | Tokens iguales, distintos, ausentes y tipos inválidos |
| ReglasVentaTest | 20 | Cantidades, stock, estado y formularios |

La matriz docs/MATRIZ_PRUEBAS.csv y su versión MD describen las condiciones, los resultados esperados y los obtenidos. Distinguen las 37 pruebas unitarias de los 10 casos de integración. Estos últimos utilizaron Node.js fetch y el cliente MariaDB y generaron 23 solicitudes HTTP; no se realizaron con Postman.

| Caso | Objetivo | Esperado | Obtenido y estado |
|---|---|---|---|
| I1 | Entrada y movimiento | Stock 10 a 15 y movimiento de 5 | Stock final: 15 unidades. Se registró una entrada de 5. Aprobada. |
| I2 | Venta transaccional | Stock 13, total 2501 y detalle 2 | Venta 7 registrada por 2501; stock final: 13 unidades. Aprobada. |
| I3 | Reversión por stock insuficiente | Sin cambios parciales tras procesar una primera línea válida | Sin cambios parciales: stock 13, 3 ventas, 3 detalles y 8 movimientos. Aprobada. |
| I4 | Producto inactivo | Rechazo servidor y datos intactos | La venta del producto inactivo fue rechazada. Aprobada. |
| I5 | Anulación por dueño | Stock 15, marca, responsable y fecha | Stock restituido a 15; responsable y fecha de anulación registrados. Aprobada. |
| I6 | Anulación duplicada | Sin segunda restitución | Segunda anulación rechazada; stock conservado en 15. Aprobada. |
| I7 | Restricción por rol | Empleada rechazada; venta vigente y stock 13 | Operación rechazada con redirección HTTP 302; stock sin cambios, en 13. Aprobada. |
| I8 | CSRF ausente y falso | 403 y ninguna escritura | Los dos intentos con CSRF ausente o falso devolvieron HTTP 403. Aprobada. |
| I9 | Indicadores de ventas vigentes | Incrementan al vender y vuelven a base tras anular | Ventas: 0 antes, 1 al vender y 0 tras anular. Total durante la venta: $2.501. Aprobada. |
| I10 | Catálogo autenticado y activo | Activos visibles, inactivo excluido, sin sesión 401 | Producto activo visible e inactivo excluido. Sin sesión: HTTP 401. Aprobada. |

Los registros mantienen las fechas de sus ejecuciones para poder identificarlas. Las capturas del editor y la terminal complementan esos resultados y no se presentan como nuevas pruebas.

## 14. Control de versiones

El repositorio de EV01 utiliza la rama main. El último commit observado es f2037ac6592807ccadec84748e6cc41daf5500d5 y su índice contiene 73 rutas, incluidas algunas de documentos que se retiraron de la carpeta de trabajo.

| Commit | Contenido |
|---|---|
| f2037ac | Documentar resultados y preparar informe PDF - GA8 AA1 EV01 |
| cf28d0e | Integrar reglas de seguridad y pruebas de modulos - GA8 AA1 EV01 |

En EV01 hay cambios de documentación, cinco documentos eliminados y evidencias que no se han incluido en un commit. El repositorio no tiene un remoto configurado. La captura 01 muestra sus commits, pero no demuestra que el instructor pueda acceder al código en línea.

EV02 todavía no tiene un repositorio Git propio. En docs/CONTROL_VERSIONES.md explico de dónde procede el código y los pasos propuestos para versionarlo, que requieren autorización. La carpeta y el ZIP permiten revisar los archivos, pero no sustituyen ese repositorio.

## 15. Limitaciones

No hay acta formal de requisitos aprobados, aceptación del cliente/evaluador ni URL pública de módulos verificada. Los espacios de firma permanecen sin completar.

Para ejecutar el sistema desde EV02 falta preparar una base propia y adaptar el ejecutor con autorización. Los casos registrados no comprueban todas las opciones de productos y reportes, ni pruebas de carga o concurrencia.

Los módulos PHP de esta entrega no necesitan React ni MongoDB. Las demás funciones conservadas en el código no cuentan con pruebas adicionales por el hecho de estar incluidas.

Se mantienen las limitaciones descritas: bloqueo de login por sesión, cálculo monetario con float y diferencias de validación y filtros entre algunas pantallas. Esta revisión de redacción no las corrige.

## 16. Conclusiones

Al relacionar los módulos con sus entradas, validaciones y salidas, puedo explicar cómo funciona el sistema y dónde se encuentra su código. Los resultados de EV01 muestran la relación entre inventario, ventas, anulación e indicadores.

Para EV02 reuní el código, las pruebas y la documentación sin cambiar su funcionamiento. Antes de entregar falta incorporar mi portada, definir el acceso al código mediante Git y revisar el acta. Los resultados técnicos no equivalen a una aceptación formal.

## 17. Anexos técnicos

| Anexo | Contenido |
|---|---|
| docs/MODULOS.md | Descripción editable de los seis módulos |
| docs/AMBIENTE_DESARROLLO_PRUEBAS.md | Dependencias y separación de ambientes |
| docs/MANUAL_TECNICO.md | Preparación y comandos condicionados |

| Anexo | Contenido |
|---|---|
| docs/MATRIZ_PRUEBAS.csv y .md | 47 registros: 37 unitarios y 10 integrados |
| docs/CONTROL_VERSIONES.md; docs/VERIFICACION_REVISION.json | Historial observado y correspondencia de fuentes |
| docs/ACTA_PRUEBAS_ACEPTACION_EV02_REVISADO.docx/.pdf | Resultados técnicos y aceptación pendiente |
| docs/resultados/ | Cinco registros auténticos reutilizados |
| evidencias/01_Control_Versiones_Git.png | Terminal VS Code con commits del laboratorio |
| evidencias/02_Pruebas_Unitarias.png | PHPUnit 11.5.57 / PHP 8.2.12 y 37/37 |
| evidencias/03_Modulo_Ventas.png | Código de venta y organización del proyecto |
| evidencias/04_Buenas_Practicas.png | Reglas de CSRF y cantidades; líneas largas ya cortadas en el editor original |
| MANIFIESTO_ENTREGA.json y SHA256_ENTREGA.txt | Inventario verificable de la entrega |

Las capturas y resultados proceden de EV01; no se atribuyen a una ejecución EV02. La aprobación académica corresponde al instructor.